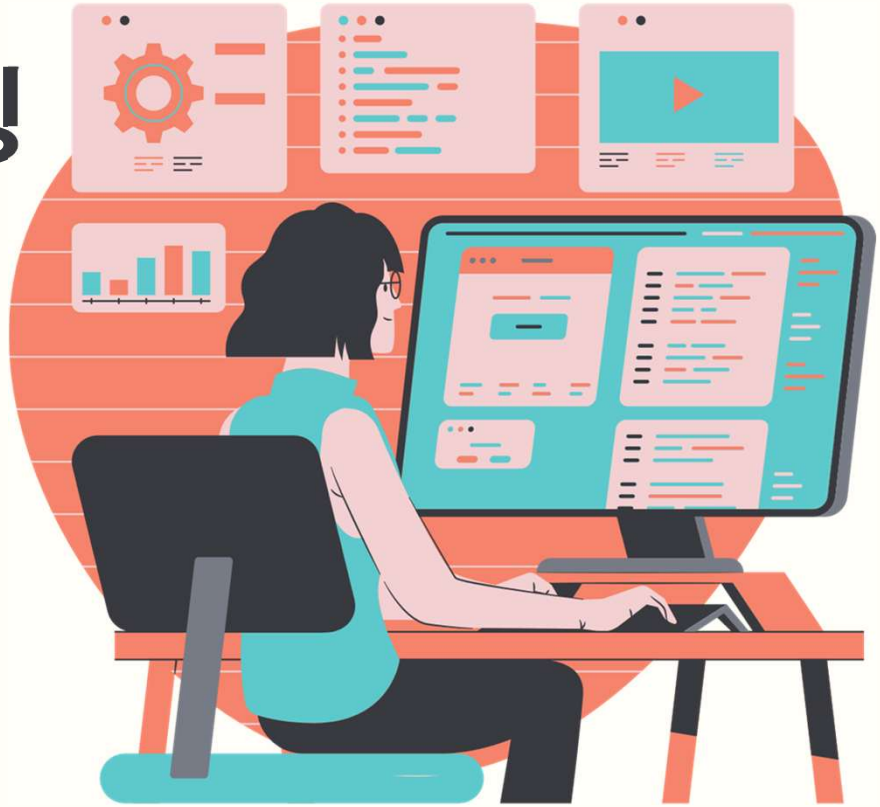
은행 시스템 구현 및 테스팅



소속: 인공지능 소프트웨어 하이테크 과정

작성자: 김근하

PRESENTATION

목차

01 요구사항 분석

02 설계(ERD, 개발 계획)

03 구현(파이썬 코드)

04 테스트(파이썬, 오라클연동)

01 요구사항 분석(SRS)

※은행 시스템 기본 구상도※

시작
└── 메인 메뉴
1. 회원가입
2. 로그인
3. 종료

로그인 성공 후
└── 은행 메뉴
1. 내 계좌 생성
2. 내 계좌 조회
3. 입금
4. 출금
5. 계좌이체
6. 거래내역 조회
7. 로그아웃

※은행 시스템 관련 기본 요구사항서(SRS)※

1. 사용자는 로그인 후 은행 계좌 관리 시스템의 서비스를 이용할 수 있다

   (은행 매뉴: 내 계좌 생성, 조회, 입금, 출금, 계좌 이체, 거래내역 조회)

2. 계좌 입금 시 최소 금액은 0원보다 커야 함.

3. 사용자는 본인 소유 계좌 외에 타인 계좌로 사용할 수 없다.

◆번 외(고객 요구 없어도 기본 암묵적 요구사항)◆

1. 로그인 이후 사용자 은행 메뉴 진입 가능 할 것.

2. 로그 아웃 이후 시작 메뉴로 돌아 갈 것.

3. 종료 이후 프로그램 종료될 것.

4. 회원가입, 계정 정보, 계좌 정보에 대한 데이터가 오라클 데이터 베이스 상에

   데이터 기록이 저장되어 있을 것.

01 요구사항 분석(SRS)

▶ 기능적 충족 사항

-. 사용자 인증 및 세션 제어 : 회원 가입 이후 로그인 하여 은행 메뉴 진입, 로그아웃 이후 초기 메뉴로 회귀

-. 핵심 은행 서비스: 사용자 계좌 관리, 금액 이동, 거래 내역 조회

▶ 비 기능적 충족 사항

-. 데이터 영속성: 데이터가 영구적으로 휘발 되지 않고 보존 되어야 함.

-. 비즈니스 규칙 및 예외 처리: 금액 값이 0이 아니어야 하고, 금액 값이 초과하는 거래 일 경우 차단 되어야 함.

-. 보안 및 권한 검증: 사용자 본인 소유의 계좌만 조회 및 제어 가능해야 함.

-. 데이터 무결성 및 트랜잭션: 금액 이동에 대한 오류가 없어야 함.

02 설계(ERD)

1. 엔티티(Entity) 및 속성(Attribute) 구성

아이디, 비번, 계좌번호, 금액 간의 연동 검토에 따른 세 개의 주요 테이블 설계

회원 테이블 (Users)

PK	속성명	데이터타입
	User_id	VARCHAR(20)
N	Password	INT

* 사용자의 로그인 정보를 관리합니다.

1 — N

계좌 테이블 (Accounts)

PK	속성명	데이터타입
	Account Number	INT
FK	User_id	VARCHAR(20)
N	Account Password	INT
N	Current Balance	INT

* 사용자가 소유한 다수의 계좌 정보를 관리합니다.

1 — N

거래내역 테이블 (Transactions)

PK	속성명	데이터타입
	Transaction_id	INT
FK	Account Number	INT
N	Type	VARCHAR(20)
N	Amount	INT
N	Target Account	VARCHAR(20)
N	Created_at	DATE

* 입출금 및 이체 등 모든 거래 이력을 기록합니다.

■ 테이블 간 관계

-. 회원 테이블 ↔ 계좌 테이블 (1:N) 관계: 1명의 회원은 여러 계좌를 가질 수 있음.

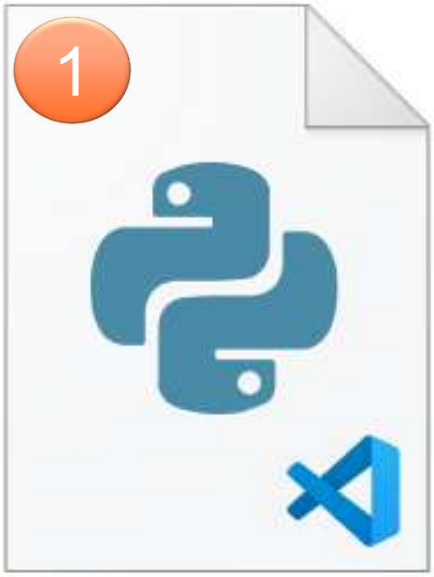
-. 계좌 테이블 ↔ 거래내역 테이블 (1:N) 관계: 1개의 계좌는 여러 거래 내역을 가질 수 있음.

■ 은행 시스템 동작 흐름

.

03 구현(파이썬 ↔ 오라클 DB 연동)

■ 파일 구성



Create Data (SQL 초기 데이터베이스 세팅용 파일)

bank_main

login_bank

03 구현(파이썬 ↔ 오라클 DB 연동)

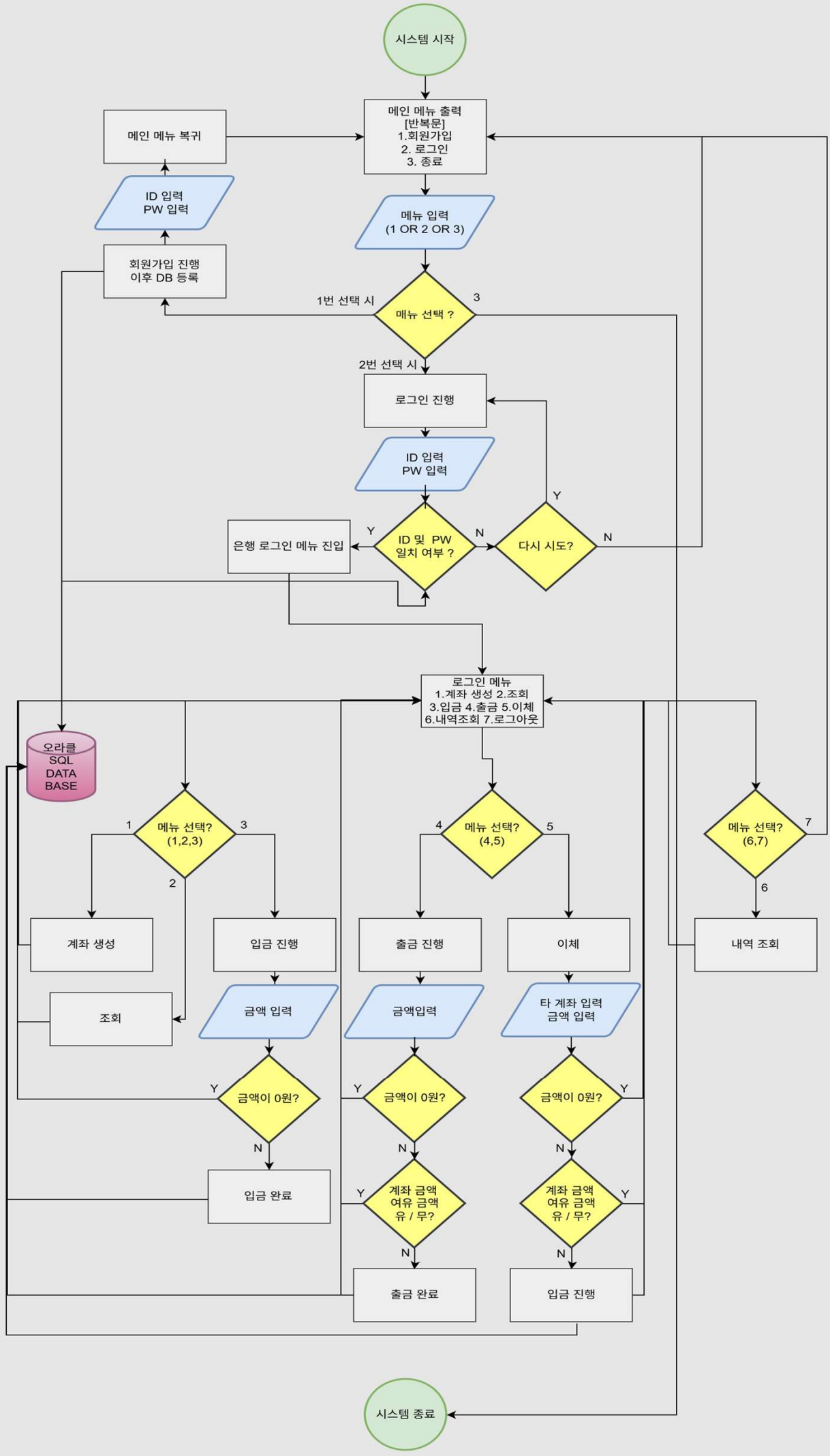
■ 오라클 DB 구축에 대한 세팅 (테이블 값 정리)

```
-- 사용자 테이블
CREATE TABLE Users (
    user_id VARCHAR(20) PRIMARY KEY,
    password VARCHAR(20) NOT NULL
);
```

```
-- 계좌 테이블
CREATE TABLE Accounts (
    account_number VARCHAR(20) PRIMARY KEY,
    user_id VARCHAR(20),
    password VARCHAR(4),
    balance INTEGER DEFAULT 0,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

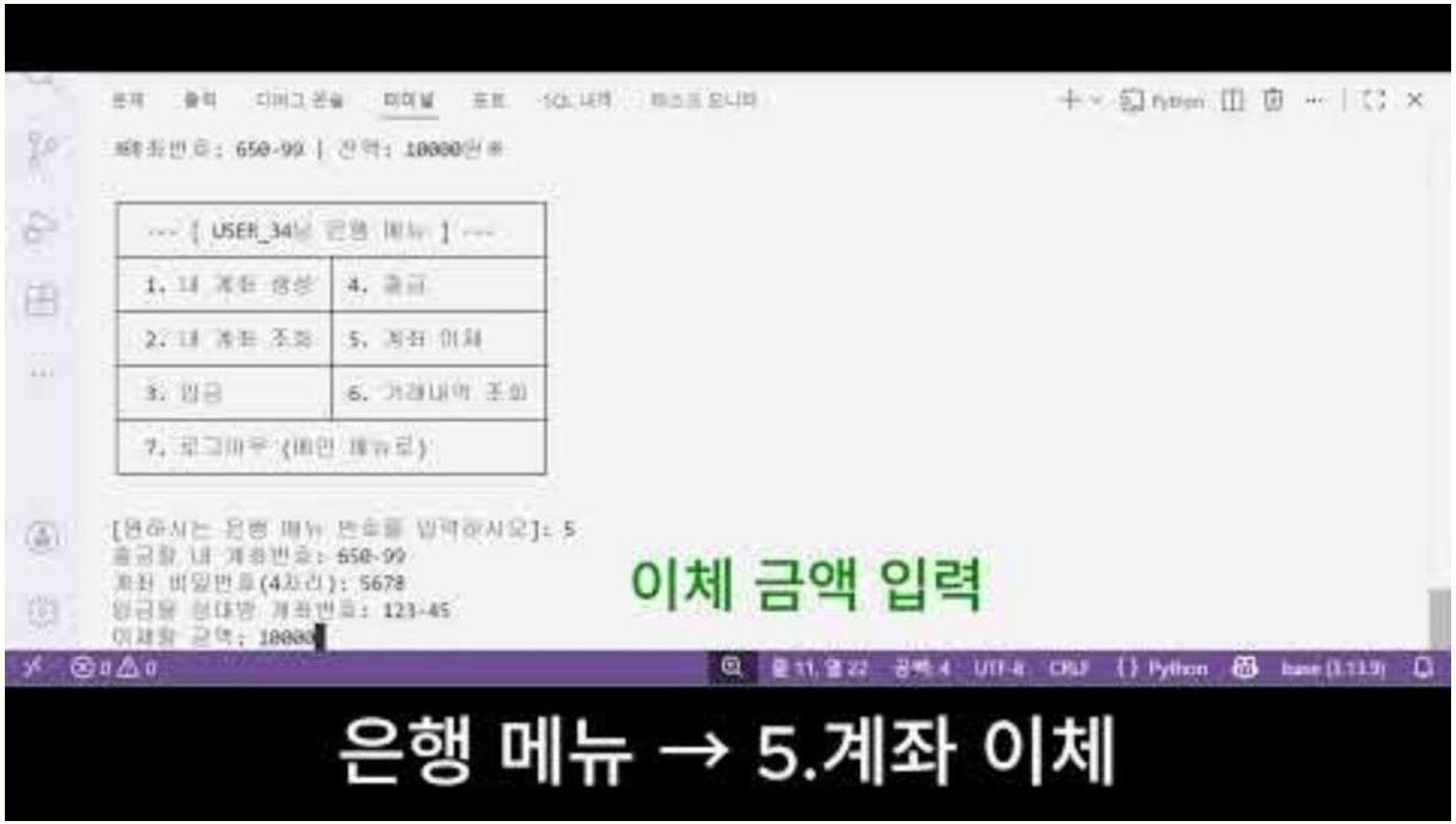
```
-- 거래 내역 테이블
CREATE TABLE Transactions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    account_number VARCHAR(20),
    type VARCHAR(10), -- 입금, 출금, 이체
    amount INTEGER,
    target_account VARCHAR(20), -- 이체 시 상대 계좌
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (account_number) REFERENCES Accounts(account_number)
);
```

은행 시스템 동작 순서도


시스템 시작
메인 메뉴 출력
[반복문]
1.회원가입
2. 로그인
3. 종료
메인 메뉴 복귀
ID 입력
PW 입력
회원가입 진행
이후 DB 등록
메뉴 입력
(1 OR 2 OR 3)
1번 선택 시
매뉴 선택 ?
3
2번 선택 시
로그인 진행
ID 입력
PW 입력
Y
은행 로그인 메뉴 진입
Y
ID 및 PW
일치 여부 ?
N
다시 시도?
N
로그인 메뉴
1.계좌 생성 2.조회
3.입금 4.출금 5.이체
6.내역조회 7.로그아웃
오라클
SQL
DATA
BASE
1
메뉴 선택?
(1,2,3)
3
2
4
메뉴 선택?
(4,5)
5
메뉴 선택?
(6,7)
7
6
계좌 생성
입금 진행
출금 진행
이체
내역 조회
조회
금액 입력
금액입력
타 계좌 입력
금액 입력
Y
금액이 0원?
Y
금액이 0원?
Y
금액이 0원?
N
N
N
입금 완료
Y
계좌 금액
여유 금액
유 / 무?
Y
계좌 금액
여유 금액
유 / 무?
N
N
출금 완료
입금 진행
시스템 종료

04 테스트 (실제 프로그램 시현 진행)

■ 구동 시작



발표를 마치며..

- 파이썬 관련하여 함수와 문장 관련하여 문제를 푸는 작업만 진행 하였다가 [은행 시스템] 이라는 큰 대목으로 하나의 시스템을 구현 하는 작업을 하는 것이 무척 생소 하였고 해당 작업을 통해서 ER 다이어그램은 왜 사용 하는지, SRS는 무엇이고 설계는 어떻게 진행하는지 등을 통하여 제 자신이 파이썬 운용 관련하여 어떤 것이 부족하고 이 과목을 왜 배우고 실습을 하는지 다시 돌아 보게 되었습니다.

- 파이썬을 운용하는 것이 다른과목에서의 SQL 이라는 데이터 운용과 합작이 되는 작업에 대해 무척 신기하고 생소 하였습니다. 어떻게 보면 소프트 웨어에서 배우는 과목들은 엮여 있다는 것을 해당 프로젝트를 통하여 물리적으로 체감 한 기분이 듭니다. 이를 통하여 한 과목을 배우더라도 좀 더 안목을 넓게 봐서 어떻게 응용하여야 하는지 생각을 많이 해봐야 겠다라는 마음 가짐이 들었습니다.

- 마지막으로, 제가 평상시에 사용하였던 사소한 시스템들 (은행 ATM, 커피 키오스크, PC 방, 전자 민원 시스템 등등) 에 대해 일반인 들이 사용하는 데 불편함이 느껴지지 않도록 수많은 소프트웨어 종사자들에 대한 노력의 결과 들이었다고 생각이 들었고, 저는 그런 분들 중 일원이 될 수 있도록 더 정진 하여 좋은 성과를 만들 사람이 되어야 겠다고 생각이 들었습니다.

감사합니다